

Can we run the mathematics in a research paper?

A practical audit of equations and reported results

Jeppe Rich

DTU Management

August 18, 2026

Some errors survive every careful reading

1. A term disappears during a long derivation.
2. A calibration no longer reproduces its own table.
3. An optimizer is asked to solve a different problem from the one described.

These errors look plausible on the page. They become obvious when the claim is executed.

Producing a result is hard; checking it is often cheap

Claim	Produce	Check
Algebraic identity	derive it	simplify left – right
Calibrated number	estimate it	substitute and compare
Feasible solution	search for it	recompute constraints
Claimed optimum	solve a hard problem	compare value and bound

Verification exploits the gap between the last two columns.

The paper becomes an executable checklist

1. Translate

Turn each equation, implication, table entry, and solver claim into a small executable check.

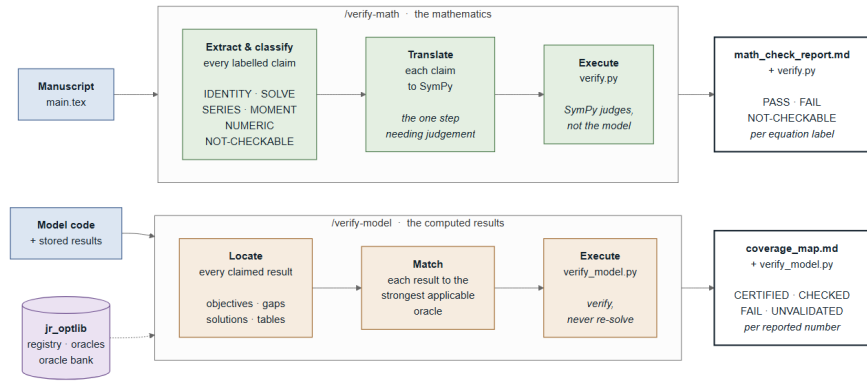
2. Judge

The computer returns **PASS**, **FAIL**, or **NOT CHECKED**. It does not decide whether an argument sounds convincing.

The rule

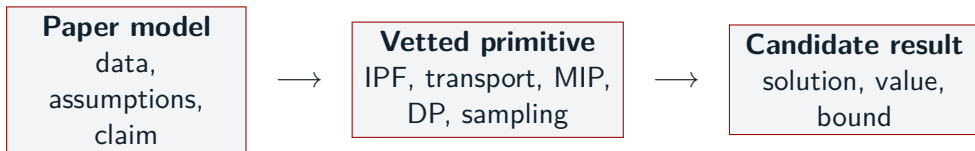
Every mathematical claim is either executed or explicitly listed as not executed.

A human translates; the machine checks



The report shows the original claim, the executable version, the verdict, and the boundary of coverage.

Inside the optimization library



Solver backend: produce

NumPy/SciPy, OR-Tools, PuLP, or a custom algorithm. **Gurobi is optional:** used for selected MIPs, MIQPs, and exact benchmark bounds.



Independent oracle: judge

Recompute feasibility and value; compare a dual bound, exact solver, brute-force instance, reference implementation, or invariant.

The harness converts the oracle outputs into **CERTIFIED / CHECKED / FAIL / UNVALIDATED**. At submission, the paper pins the library commit.

First test: one of our own papers

58

executable checks

- 55 passed immediately.
- Three calibrated percentages failed.
- All three failures traced to one stale calibration.
- Correcting one input restored all three results.

The equations were not difficult. The problem was that nobody had substituted the final parameters back into them.

Then we took three papers at random

- Three newly published papers were supplied without screening.
- All appeared in leading transportation journals.
- We checked the papers as printed, not the authors or their reputations.
- Names are withheld: the point is the failure mode, not attribution.

Important boundary

These are internal-consistency findings. Unavailable source code could silently correct a printed formulation.

Anonymous paper A: the objective changes direction

The text asks the model to

minimize pollution and maximize parking flow.

But the printed optimization problem is

$$\min (J_{\text{pollution}}, J_{\text{parking}}),$$

and the weighted solver gives both quantities positive weights.

What goes wrong

The optimizer is rewarded for reducing parking flow—the opposite of the stated policy objective. The reported table confirms that parking flow is stored as a positive quantity, not as a loss.

Anonymous paper B: variance is used as a scale

The method says that the scaling matrix contains **standard deviations**. The example states a variance of 4, but inserts 4 rather than $\sqrt{4} = 2$.

	Scale used	Reported metric
Printed example	4	11.52
Consistent with variance 4	2	12.50

The same paper's convergence proof first derives $r^* \geq r$ and then concludes $r^* < r$ from the same premise.

Arithmetic independently recomputed; the algorithm may still be correct despite the broken printed proof.

Anonymous paper C: objective and algorithm diverge

The stated objective is an undiscounted sum:

$$\mathbb{E} \left[\sum_{t=0}^T r_t \right] .$$

The Bellman equation and experiments instead discount the future:

$$V(s_t) = \max_a \mathbb{E}[r_t + 0.9 V(s_{t+1})] .$$

A second formulation gap

A relocation-time constraint applies to every possible destination, even when that destination is not selected. It is missing the condition “if this relocation is chosen.”

Four papers: what passed, and what was flagged

Paper	Correct checks observed	Flags
Anonymous A	5/5 numerical spot checks	1 objective-direction inconsistency
Anonymous B	Printed arithmetic reproduced	1 variance/scale inconsistency; 1 contradictory proof step
Anonymous C	2/2 accounting identities	2 formulation gaps: discounting and relocation condition
Famous benchmark	49/49 derivations	1 unstated side condition; 3 strictness gaps

The famous-paper row is a complete derivation audit. The anonymous rows are targeted internal-consistency screens, so their counts are not coverage totals.

A famous stress test: prospect theory

We also transcribed every stated derivation in a landmark economics paper and ran them end to end.

Checks run	49
Passed	49
Failed	0
Unstated side condition found	1
Strictness gaps	3

The result is not “machines find mistakes everywhere.” It is that they distinguish sound derivations from the few steps that need an extra condition.

What this does—and does not—establish

It can establish

- identities and implications;
- numerical consistency;
- feasibility and objective values;
- explicit coverage and omissions.

It cannot establish

- that the translation is faithful;
- that assumptions are realistic;
- generality from a few examples;
- a formal proof of the whole paper.

This is an audit of stated mathematics, not formal verification.

Why the modest version matters

- Formal proof systems offer stronger guarantees, but require specialist effort.
- This approach works with the paper researchers already write.
- Failures point to a specific equation, table entry, or assumption.
- The cost is low enough to run before every submission.

A limited check that runs on every paper can prevent more errors than a perfect check that is never attempted.

Execute it.

1. Turn every checkable claim into an executable statement.
2. Let the machine judge equality, feasibility, and arithmetic.
3. Report both what passed and what was never checked.

The goal is not perfect certainty. It is fewer avoidable errors.